

Why Pay for Long Reasoning When Rewrite Do Trick?

Causal Effects of Reasoning History Compaction on Future Generation and Agent Cost

Anton Hristov

hristov.anton@icloud.com

Abstract

Reasoning substantially improves language models’ ability to solve difficult problems, but long reasoning traces make these gains expensive. Efficient reasoning should express useful analysis in fewer tokens by reducing repetition and text that adds little value to solving the problem. Progress in proprietary models motivates this goal, yet their hidden scratchpads limit direct study of how it is achieved. We investigate whether open-weight models can generate less verbose reasoning by changing the form of their own earlier thoughts, without retraining. Our intervention rewrites prior reasoning into compact state records before pending tool results are revealed, then compares continuations from original and compact histories. Across the tested models, accepted compact histories usually shorten future reasoning; a coding case study also shows less repeated verification after a shared candidate solution. In software-agent deployments, compaction can reduce total spending, although task quality and rewrite-inclusive time depend on the target and compressor. These findings identify reasoning history as one way to reduce future reasoning-token use and motivate model-specific compaction policies evaluated for both efficiency and retained capability.

1 Introduction

Reasoning is a major source of language-model capability: generating intermediate steps improves problem solving, and recent reasoning models extend these gains to complex coding and tool use (Wei et al., 2022; OpenAI, 2026a). Those gains carry a substantial inference cost. In Artificial Analysis data retrieved on August 3, 2026, reasoning accounts for 84.9% of generated tokens for GLM 5.2 and 75.1% for Claude Fable 5. For DeepSeek V4 Pro, reasoning output alone accounts for 53.4% of total API cost (Artificial Analysis, 2026). Reasoning can therefore dominate

generated output, while agents also repeatedly pay to process earlier scratchpads as input history.

The goal of efficient reasoning is to retain the analysis needed to solve a problem while reducing repeated analysis and low-value tokens. OpenAI reports that GPT-5.5 improves coding performance over GPT-5.4 while using fewer tokens (OpenAI, 2026a). Practical inspiration for this study came from occasional, unofficial exposures of GPT-5.5’s reasoning scratchpad, which appeared compact. These observations are anecdotal, and the scratchpads are not reliably accessible for controlled study: OpenAI does not officially expose raw reasoning (OpenAI, 2026b). We therefore study open-weight models whose reasoning is accessible for observation and controlled rewriting.

Can these models be induced to use less reasoning without new training? We test one approach: rewrite their earlier scratchpads as concise state records and let the same model continue naturally. Prior reasoning may serve as an in-context example of how concisely to express subsequent analysis. Changing its form could therefore reduce newly generated reasoning, beyond the immediate saving from shortening the input. The rewrite is instructed to preserve decisions, uncertainty, and intended actions; the target receives no new instruction to be brief.

Concise demonstrations and surrounding context already influence reasoning length (Jin et al., 2024; Renze and Guven, 2024; Xu et al., 2025; Ge et al., 2025). Reasoning Shift shows that added context can shorten reasoning and verification, sometimes at a quality cost (Rodionov et al., 2026). Related approaches manage agent context (Sun et al., 2025; Lian et al., 2026; Li et al., 2026) or steer concise reasoning states (Azizi et al., 2026; Chen et al., 2025). We isolate a rewrite of the model’s own earlier reasoning while preserving other messages and withholding future tool results

from the compressor.

We first test whether this intervention changes subsequent generation in paired continuations, then evaluate continuous compaction in software agents. The controlled results show frequent shortening across model families, while deployment quality and economics vary by target and compressor. Compaction is thus a candidate route toward more efficient reasoning, with practical value that must be assessed per model rather than inferred from shorter traces alone.

2 Method

Reasoning-only compaction. A compressor LLM converts historical assistant scratchpads into shorter state records. It is instructed to retain decision-relevant facts, numbers, uncertainty, conclusions, and intended actions while removing repetition. System and user messages, tool calls, arguments, and observations remain intact. The target’s weights, tools, and current instructions are unchanged. We call the original history *clean* and the rewritten history *compact*. Code and frozen experimental records for reproducing the analyses are released at <https://github.com/DebelToni/ShortReasoning>.

Outcome-blind timing. During shared acquisition, each rewrite completes after an action is requested and before its result is executed or revealed. At the fork, the histories are frozen and independently continued under the same model/provider route and settings (Figure 1). Serialized first requests are checked field by field to verify that only assistant reasoning differs. Consequently, the first-turn contrast measures the effect of the history intervention after a shared observation; later contrasts can also reflect treatment-induced trajectory changes.

Acceptance and estimand. Routes must demonstrate functional use of replayed reasoning. Action and fidelity gates select sources before continuation. GPT 5.6 Sol writes rewrites and judges them in separate rubric-based calls. The estimate is conditional on accepted treatment histories (Appendix A).

We measure provider-exposed future reasoning tokens. For clean count R_C and compact count R_K , we report $100(R_K/R_C - 1)$; negative values indicate reduction. Token comparisons use completed pairs with equal postfork turn counts. This

Route	Sources retained	Compact shorter	Median Δ [95% CI]	Success clean→compact
DeepSeek V4 Flash	12/12	30/36	-29.5% [-39.5, -16.5]	36/36→35/36
Laguna S 2.1	12/12	32/36	-46.2% [-56.3, -29.6]	36/36→35/36
MiniMax M3	10/12	25/30	-27.3% [-38.4, -17.1]	30/30→30/30
GLM 4.7 Flash	5/12	13/13	-50.5% [-66.6, -40.1]	11/15→14/15
GLM 5.1	10/12	28/30	-20.2% [-28.6, -12.5]	25/30→25/30
DeepSeek V4 Pro	9/12	23/27	-32.6% [-43.4, -12.2]	27/27→27/27

Table 1: Retrospective retry-augmented results on accepted histories. Shorter counts use equal-horizon pairs; success includes all three replicates per source. Intervals use 10,000 source-level bootstrap resamples. Retention differs across routes, precluding a matched-task model ranking.

endpoint-dependent matching can select on a treatment consequence. Exact success retains every included pair. Confidence intervals resample source histories with their continuation replicates.

3 Experiments

3.1 Controlled Continuations

Effect on subsequent reasoning. We test whether compacting earlier reasoning changes how much reasoning the model generates later on. Six model/provider routes each attempt 12 multi-stage tasks with deterministic tool responses and exact final-answer checks. For example, a rescue-planning task requires choosing a plan under cost, site-count, reliability, and risk constraints as tools reveal simulations and a superseding weather update. A source history is one model’s partially completed run on one task. Precontinuation gates retain 58 of 72 model-specific source histories: five fail action checks and nine fail fidelity checks. Three paired continuations per accepted source yield 174 quality pairs and 172 comparable token pairs (Table 1). Three length-capped clean continuations and one compact provider-error continuation were rerun; Appendix A.6 reports the retry protocol and first-attempt sensitivity.

Compaction shortens 151/172 comparisons (87.8%), with route medians between -20.2% and -50.5%. Twenty pairs grow longer and one is unchanged (Figure 2); the median absolute change is -178 tokens, with a maximum increase of 2,807. Exact success changes from 165/174 to 166/174; GLM 4.7 Flash gains three successes, DeepSeek V4 Flash and Laguna each lose one, and other routes have unchanged totals. Only the most heavily filtered route gains. The evidence supports frequent shortening within accepted histories, with route-dependent quality changes.

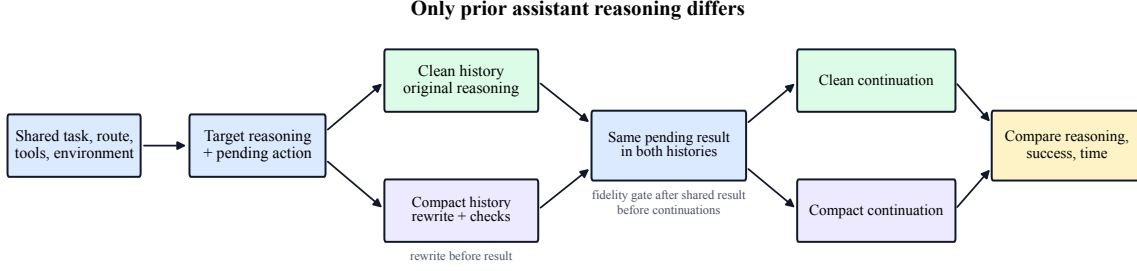


Figure 1: Outcome-blind paired fork. The target requests an action, its reasoning is rewritten before the result is available, and both histories receive the same result. Fidelity acceptance is required before continuation sampling. Only prior reasoning differs at the first continuation; later actions and observations can diverge.

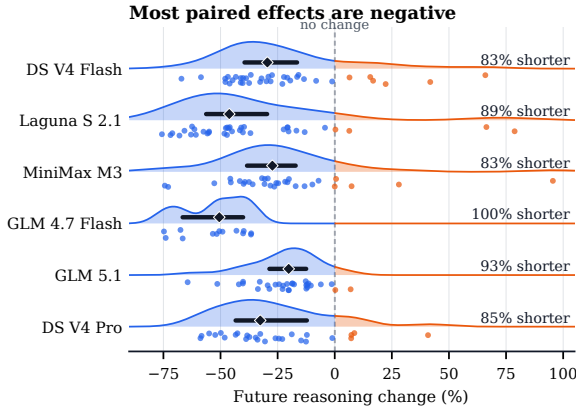


Figure 2: Paired future-reasoning changes across the six routes (172 comparable pairs). Points show continuations; diamonds and bars show medians and source-clustered 95% intervals. Blue marks shortening; orange marks increases or equality. DS denotes DeepSeek.

Rewrite properties and length-matched controls. To test whether generic rewording elicits shorter continuations, we compare original histories with fresh paraphrases constrained to 90–110% of their word count. Across 65 comparable pairs, median future reasoning changes by +0.7% (95% CI $[-8.2, +9.7]$), while exact success falls from 68/69 to 59/69. The near-zero median change gives little evidence of systematic shortening from this paraphrase intervention. Its lower success rate leaves the specific contribution of history length unresolved. A separate exploratory style sweep jointly varies length, syntax, wording, and compressor provenance (Appendix C); the shortening mechanism remains unresolved.

Verification behavior in a coding case study. To examine checking after a candidate solution is available, we compare clean and compact con-

tinuations with the same recorded program and identical public-test feedback. We measure reasoning on the submission turn. In the Live-CodeBench screen (Jain et al., 2024), extended verification, defined retrospectively as at least 350 reasoning tokens, occurs in 61.1% of successful DeepSeek clean branches and none of the compact branches. This illustrates shorter deliberation before submission in a selected successful setting (Appendix B.2).

3.2 Coding-Agent Deployment

Setup and accounting. To test whether shortening is useful over complete agent runs, we compare continuous compaction with normal history on a fixed 50-task SWE-bench Verified Mini sample (Jimenez et al., 2024). Four targets use either an external compressor, GPT 5.6 Luna, or self-compaction. Before each target request, we compact and cache new reasoning blocks using only their text and action name. Blocks of at most 12 words stay unchanged, and non-shorter rewrites revert to the original. All tasks remain, without fidelity-based exclusions. We evaluate each task’s final code patch with the official benchmark tests. We use the first completed agent run, including runs stopped by agent limits. Infrastructure failures are retried.

Spend includes all captured completed target and compressor model responses, including those absent from saved trajectories. The main table isolates target request time to characterize changes in the agent’s work; Appendix D.5 adds compressor time and separately accounts for JSON error bodies. Request durations include queueing and network transfer; tools, orchestration, and evaluation are excluded. Nemotron spend uses common tariffs across free and paid providers. These comparisons retain provider and retry variation.

Target	Compressor	Score	Unique reasoning-history blocks				Change from Normal (%)		
			Raw words	Compact words	Saved words	Reduction	Spend	Tool calls	Target time*
DeepSeek V4 Flash	External	32/50	170,725	84,877	85,848	50.3%	-40.6	-19.9	-40.5
	Self	29/50	181,155	77,922	103,233	57.0%	-43.6	-29.8	-41.5
MiniMax M3	External	31/50	193,573	70,808	122,765	63.4%	-41.2	-23.6	-72.4
	Self	30/50	142,601	53,857	88,744	62.2%	-51.1	-24.8	-76.5
Nemotron 3 Ultra	External	32/50	269,284	146,786	122,498	45.5%	+60.9 [†]	+7.7	-1.2
	Self	29/50	374,962	165,578	209,384	55.8%	+109.7 [†]	+0.7	+28.3
Kimi K2.6	External	32/50	753,729	236,549	517,180	68.6%	-7.6	+19.8	-21.7
	Self	31/50	520,930	118,701	402,229	77.2%	-31.9	-4.6	-57.8

Table 2: Continuous compaction over 50 assigned tasks. Normal scores are 32/50, 30/50, 29/50, and 31/50 for DeepSeek, MiniMax, Nemotron, and Kimi. History words count each block once; tool calls describe selected terminal trajectories. Spend includes compressor calls and abandoned acquisition attempts; [†] marks common-tariff estimates. *Target time sums target requests only, excluding rewriting, tools, and orchestration. Rewrite-inclusive timing is in Appendix D.5.

Results and practical interpretation. Table 2 shows accuracy changes ranging from -6 to +6 percentage points. Spend falls in six arms and rises in both common-tariff Nemotron comparisons. Inclusive request time falls in five arms and rises in three. In particular, DeepSeek self-compaction changes from a 41.5% target-only time reduction to a 78.3% inclusive increase (3.76 to 6.70 summed request-hours). Across all eight arms, 63.4% of unique historical reasoning words are removed. These counts describe unique stored reasoning blocks. Overall expenditure also depends on replay, compressor overhead, and subsequent agent work. Paired task-bootstrap intervals (Appendix D.4) quantify task-sample variation conditional on the captured runs.

To understand differences in agent work, we retrospectively count tool calls in the selected terminal trajectories, excluding abandoned attempts. Changes range from -29.8% to +19.8%. Kimi with external compaction illustrates the combination of more steps and shorter reasoning: selected assistant responses rise from 1,842 to 2,304 (+25.1%), while mean reasoning per response falls 31.1% and total reasoning falls 13.8%. This arithmetic decomposition explains the total change through response frequency and reasoning length per response.

Preprocessing amortization and single-task marginal latency are analyzed separately in Appendix E; the hosted one-use example loses both time and money.

4 Discussion

Our controlled comparisons show that changing how earlier reasoning is written can reduce how much reasoning a model generates next. The tar-

get receives no new instruction to be brief. We still do not know which changes cause the shortening. Rewrites may remove repetition, emphasize different facts, or lose important doubts. Changes in later actions can also increase or reduce the eventual savings.

For software agents, the goal is to solve tasks reliably with less total work. Shorter reasoning helps when the savings exceed the cost of rewriting and any additional work the agent performs afterward. Because these tradeoffs differ across models, useful compaction policies must be evaluated for task success, total cost, and time spent on both rewriting and task completion.

Our deployment rewrites new reasoning blocks before each target request. The compressor sees only the block and its action name. Future work should test rewriting less often, such as every N agent turns, and giving the compressor more of the conversation. This additional context may help preserve important information. When the target also performs compression, shared conversation prefixes may allow reuse of cached computation. These alternatives should be compared at similar task quality, counting the cost and time of rewriting.

5 Conclusion

Our goal is to express useful reasoning in fewer tokens through a more compact style. We test whether rewriting earlier scratchpads can encourage this behavior without retraining. We find frequent shortening among accepted histories and cost savings in some software-agent deployments. Further work should examine how rewriting affects later token use, when to rewrite, and what context the compressor needs.

Ethical Considerations

External rewriting can expose sensitive history to another provider. Applications should minimize data, redact secrets, protect verbatim fields, and use trusted or local compressors. Potential loss of uncertainty or verification state warrants task-specific quality checks.

Limitations

The controlled tasks share a narrow structure, source acceptance is asymmetric, and the same model writes and judges rewrites without independent human validation. Equal-horizon selection limits trajectory-wide inference. We lack a matched deletion, truncation, or extractive-retention baseline, so comparative value over free alternatives remains untested. The deployment uses one nonrandom sample without repeated seeds; mixed providers further limit causal interpretation. The style and coding studies are exploratory. Visible reasoning tokens and client-observed request durations leave hidden computation and complete operational latency outside the measurement scope. Quality noninferiority remains unestablished. The influence of training-data diversity on model-family compatibility remains untested (Appendix D.3).

References

- Artificial Analysis. 2026. [Artificial analysis intelligence index v4.1](#). Online benchmark dataset. Data retrieved August 3, 2026.
- Seyedarmin Azizi, Erfan Baghaei Potraghloo, Souvik Kundu, and Massoud Pedram. 2026. [Activation steering for chain-of-thought compression](#). In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 36676–36687.
- Runjin Chen, Zhenyu Zhang, Junyuan Hong, Souvik Kundu, and Zhangyang Wang. 2025. [SEAL: Steerable reasoning calibration of large language models for free](#). In *Conference on Language Modeling (COLM)*.
- Yuyao Ge, Shenghua Liu, Yiwei Wang, Lingrui Mei, Lizhe Chen, Baolong Bi, and Xueqi Cheng. 2025. Innate reasoning is not enough: In-context learning enhances reasoning large language models with less overthinking. *arXiv preprint arXiv:2503.19602*.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fan-jia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Live-CodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [SWE-bench: Can language models resolve real-world GitHub issues?](#) In *International Conference on Learning Representations*.
- Mingyu Jin, Qinkai Yu, Dong Shu, Haiyan Zhao, Wenyue Hua, Yanda Meng, Yongfeng Zhang, and Mengnan Du. 2024. [The impact of reasoning step length on large language models](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 1830–1842.
- Tianjian Li, Jingyu Zhang, William Jurayj, Xi Wang, Chuanyang Jin, Mehrdad Farajtabar, Eric Nalisnick, and Daniel Khashabi. 2026. [Self-compacting language model agents](#). *Preprint*, arXiv:2606.23525.
- Shuquan Lian, Juncheng Liu, Yazhe Chen, Yuhong Chen, and Hui Li. 2026. SWE-AGILE: A software agent framework for efficiently managing dynamic reasoning context. *arXiv preprint arXiv:2604.11716*.
- OpenAI. 2026a. [Introducing GPT-5.5](#). Model announcement, April 23.
- OpenAI. 2026b. [Reasoning models](#). API documentation. Accessed September 11, 2026.
- Matthew Renze and Erhan Guven. 2024. [The benefits of a concise chain of thought on problem-solving in large language models](#). In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pages 476–483.
- Gleb Rodionov, Roman Garipov, and George Yakushev. 2026. Reasoning shift: How context silently shortens llm reasoning. *arXiv preprint arXiv:2604.01161*.
- Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2025. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). In *Advances in Neural Information Processing Systems*, volume 35.
- Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. 2025. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*.

A Controlled Protocol and Fidelity Checks

A.1 Task structure and fork delivery

The deterministic tasks require successive actions and decisions under hard constraints, updated measurements, and tie-breaking rules. Each source contains three shared acquisition turns. A reasoning rewrite is generated before the corresponding acquisition tool executes. Clean and compact histories therefore share the acquisition actions and observations, with differences restricted to assistant reasoning. Continuations are sampled only after source acceptance and history freezing.

Provider-specific historical fields are stripped so the original reasoning cannot survive beside rewritten plaintext. In controlled forks, both arms use the same assistant reasoning field, and all other first-request fields match. Arm order is deterministically shuffled from the frozen continuation seed. Hosted routes can remain nondeterministic despite identical seed settings. Action-recovery checks qualify routes by demonstrating functional use of prior reasoning.

The six routes are DeepSeek V4 Flash/DeepInfra/xhigh, Laguna S 2.1/Poolside/high, MiniMax M3/DeepInfra/high, GLM 4.7 Flash/DeepInfra/high, GLM 5.1/Z.AI/high, and DeepSeek V4 Pro/Together/xhigh. All use an 8,192-token completion cap without provider fallback. Main Table 1 uses a retrospective retry-augmented synthesis. Its replacements comprise three clean Laguna S 2.1 branches that reached the 8,192-token cap: grid decision replicate two, manufacturing recall replicate three, and rescue decision replicate two. A fourth replacement retries the compact rescue replicate two after a provider-502 response. The rule permits up to 20 attempts per turn, resending the same captured request and extending a recovered branch thereafter. The recovery runs use 3, 3, 5, and 3 request attempts, respectively, including later turns but excluding the original captures.

The three cap events are observed model outcomes under the stated budget; we have no independent evidence that they were infrastructure faults. Retrying them selects different continuation outcomes and changes the estimand. The first-attempt sensitivity below therefore retains all four original branches. Correct reasoning followed by a nonexact final string still fails the original strict parser in both summaries.

A.2 Acceptance and attrition

All routes attempt the same 12 tasks, but retain model-specific histories. DeepSeek V4 Flash and Laguna retain 12 each, MiniMax M3 retains 10, GLM 4.7 Flash retains five, GLM 5.1 retains 10, and DeepSeek V4 Pro retains nine. Five source-action failures and nine fidelity exclusions account for the 14 omitted histories in the final six-route synthesis. Every accepted source contributes three continuation pairs, including failures and length increases. Two unequal-turn-count pairs from GLM 4.7 Flash remain in the 174 quality pairs but are excluded from the 172 comparable token pairs.

These final retention counts do not include every superseded development artifact. During initial source acquisition, three rewrites failed semantic review: two changed estimate status to binding-quote status, and one added a spurious threshold when a detector mistook a phase number for a constraint. Corrections made after the corresponding tool result had executed were archived and excluded from continuation sampling, including corrections prompted with the result withheld. After fixing the detector and evidence-status wording, the affected sources were reacquired. A further ambiguous manufacturing source was rejected before a later reacquisition passed. Seven superseded artifacts are recorded in the initial exclusion manifest.

A.3 Rewrite instructions and retries

The initial acquisition instructions are reproduced in Appendix C alongside the telegraphic histories reused in the style sweep. Additional-route prompts use the evidence-status variant there, inserting the following instructions immediately after its evidence-status sentence:

The requested tool call and its arguments are preserved separately outside the scratchpad. Treat them only as context: retain the intended next action, but never import facts, code, or argument details absent from the raw reasoning. Aim for at most half the raw word count when the state permits, and never make the scratchpad longer. Preserve the raw notation rather than translating mathematical expressions into implementation syntax or otherwise improving them.

The rewriter receives raw reasoning and the requested tool function and arguments. It can make up to three draft attempts before that tool executes. Empty drafts raise the completion cap; missing required candidate identifiers or hard-threshold values trigger a repair request containing the raw

text, previous draft, and required fields. Failure after three drafts stops that source. This mechanical retry is distinct from later semantic review. The additional-route configuration permits no post-capture repair or replacement source after rejection. Its durable transport permits at most two retries for retryable returned HTTP errors; ambiguous unreturned requests are not automatically resent.

A.4 Review procedure and rubric

GPT 5.6 Sol produces the controlled rewrites and reviews them in separate calls. The review payload contains the raw reasoning, candidate state, and requested tool function and arguments. Future results and continuation outcomes are excluded from that payload. The rewrite itself precedes tool execution; source-level review and acceptance precede continuation sampling. These are distinct timing requirements.

Mechanical checks verify source hashes, reasoning-only differences, requested actions, candidate identifiers, and hard thresholds. The reviewer checks material preservation and leakage. A source must have shorter aggregate reasoning and pass all applicable checks. Identity blocks need no semantic rewrite review. Model-based acceptance cannot establish independent semantic equivalence: the generator and judge can share blind spots. We inspected accepted rewrites, but performed no blinded human agreement study.

The following is the exact semantic-review system prompt used for the additional-route acquisition. The response schema is part of the prompt.

Audit one compact reasoning-state rewrite before its pending tool result exists. Pass only if it preserves every decision-relevant fact, number, uncertainty, conclusion, rejected alternative, evidence-status distinction, and intended next action from the raw reasoning; adds no unsupported fact; and neither states nor implies that the requested tool result has occurred. The tool call is separately preserved and is supplied only to check action consistency. Return only one JSON object with exactly these keys: verdict ('pass' or 'fail'), critical_or_material_issue (boolean), future_result_leakage (boolean), requested_action_preserved (boolean), explanation (short string).

Acceptance requires a pass verdict, no material issue, no future-result leakage, and preservation of the requested action. Malformed or failed review responses do not pass. The prompt's reference to

a nonexistent pending result describes the state being audited; the reviewer is given no such result. We make no claim that this model-based procedure replaces an independent annotation protocol.

Treatment example. A 206-word source turn repeatedly checks rescue constraints. Its 48-word rewrite begins: "Static checks: all R1–R5 have $\text{estimated_usd} \leq \$1,000,000$. R1 fails $\text{minimum_sites} \geq 2$ (1 site); R2 (3), R3 (2), R4 (3), and R5 (2) pass." It retains unknown reliability and risk and the exact simulation call. This illustrates the intended transformation in one source history.

A.5 Estimation and distributions

Controlled route and style intervals use 10,000 source-level bootstrap resamples with replacement. All replicates from a sampled model–task source remain together; intervals are the 2.5th and 97.5th percentiles. This resampling unit does not preserve cross-route dependence from shared task definitions. Replicates measure continuation variability within the same source history. Route-specific retention and small source counts restrict population interpretation.

Figure 2 displays each comparable pair to expose variation hidden by route medians. Most mass is below zero on every route, but positive tails remain. GLM 4.7 Flash shortens in all 13 comparable pairs and also has the lowest retention rate. The shortening estimate is conditional on route-specific source acceptance.

A.6 First-attempt and retry-augmented sensitivity

To test sensitivity to retrospective retries, an offline audit re-extracts the original branches of every accepted source and compares them with the retry-augmented cohort. Here first-attempt means the original captured continuation before the four retrospective replacements. Acquisition and transport retries within those captures remain included.

First-attempt captures contain 169 completed equal-horizon pairs, of which 149 shorten, with a paired median reasoning change of -32.1% . Only Laguna's route-level results change: its original 33 comparable pairs have median -47.1% and exact success $33/36 \rightarrow 34/36$; all other routes retain the values in Table 1. Across all 174 pairs, first-turn reasoning changes by -20.7% at the paired median, and pooled full-trajectory reasoning falls

38.1%. Table 3 retains capped and failed branches. The qualitative shortening survives. Retry augmentation changes failure counts and workload by replacing three length-capped outcomes and one provider error.

The retry-augmented first requests satisfy the reasoning-only invariant and share the preceding result in all 174 pairs. First-turn reasoning has a -20.0% paired median and a -16.4% pooled change; subsequent reasoning has a -37.8% paired median and a -40.0% pooled change. Later observations are also equal in all 173 pairs reaching turn two in both arms and all 172 reaching turn three. Later generation nevertheless follows treatment-affected text and actions, so these contrasts can contain mediated effects.

Across the 172 equal-horizon pairs, the median reasoning change is -32.0% (source-clustered 95% CI $[-38.8, -24.3]\%$), and the median absolute change is -178 tokens (CI $[-224, -126]$). Resampling the 12 shared task definitions instead, retaining all accepted routes and replicates within each sampled task, gives a median-change interval of $[-37.4, -26.0]\%$. This sensitivity preserves cross-route task dependence and observed gate missingness. The largest increase is 2,807 tokens. Table 3 retains unequal-horizon pairs and the failures remaining after retry augmentation. Reasoning falls in aggregate even without horizon matching. These retry-augmented totals exclude the discarded retry attempts.

To assess whether one route drives the result, we omit each route in turn (Table 4). The median token change stays negative throughout, between -36.7% and -29.2% . Omitting GLM 4.7 Flash reverses the quality change from a net gain to a net loss. Thus shortening is robust to this route omission check, whereas the aggregate quality sign is not. These source-level summaries do not estimate new routes, rejected sources, or unobserved provider and seed variation.

B Exploratory Controls and Behavioral Case Study

B.1 Length-matched and sentence-style controls

To test whether rewording alone reproduces shortening, the length-matched control uses fresh clean and paraphrased continuations from pre-outcome frozen variants. Paraphrases retain 90–110% of clean word count. Of 65 comparable pairs, 31

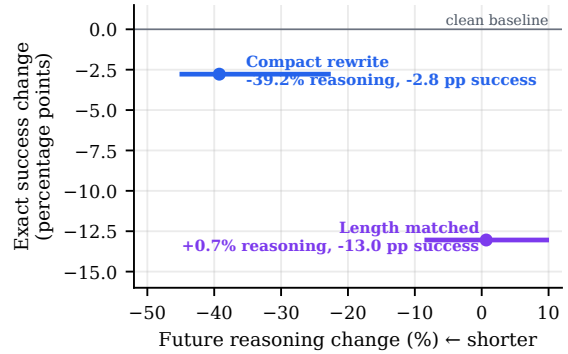


Figure 3: Length and quality control. Horizontal position is paired median future reasoning change, with source-clustered 95% intervals; vertical position is exact-success change. The paraphrase control is quality-degrading and therefore does not isolate history length.

shorten; the median change is $+0.7\%$, with source-clustered 95% CI $[-8.2, +9.7]\%$. Exact success falls from 68/69 to 59/69. For comparison, the retry-augmented two-route cohort has median -39.2% across 72 pairs, with interval $[-44.8, -22.9]\%$ (Figure 3). These are separate cohorts, and the control’s quality loss prevents attributing the contrast solely to history length.

To probe syntax within compact histories, another fresh comparison replaces telegraphic states with full sentences. Of 60 pairs, 33 shorten (55.0%); median change is -4.7% , with interval $[-16.9, +9.2]\%$, and exact success changes from 58/60 to 56/60. Nineteen parent sources are shared by both control studies. On that common-source subset, paraphrase and sentence-conversion medians are -2.4% and -5.9% , respectively, and both intervals include zero. These controls do not identify a single linguistic cause: syntax and lexical realization can also alter salience.

B.2 Selected contest-code continuations

To examine what shorter reasoning can contain, we use a LiveCodeBench v6 screen with DeepSeek V4 Flash and Laguna S 2.1. Shared acquisition succeeds for 12/24 model–item sources; 12 length-truncated failures are retained in the acquisition record and are not resampled. The accepted sources yield 36 continuation pairs. Only 33 have comparable horizons, and these are also the successful pairs. The screen is therefore conditional on acquisition and a favorable continuation regime.

Table 5 gives token and quality outcomes.

Assigned-cohort quantity	First-attempt captures		Retry-augmented captures	
	Clean	Compact	Clean	Compact
Reasoning tokens	152,475	94,357	135,778	97,790
Accumulated input tokens	1,399,681	1,259,755	1,429,111	1,277,229
Completion tokens	234,817	159,887	220,049	163,876
Continuation turns	515	518	521	520
Exact failures	12	9	9	8
Action-failure terminations	4	2	1	1

Table 3: All 174 pairs in each cohort definition, without horizon filtering. First-attempt captures retain cap events and the provider failure; retry-augmented columns substitute recovered branches and exclude discarded retry work. Input sums repeated contexts; completion includes reasoning and visible answer/tool output. Action failures are included among exact failures.

Omitted route	Shorter/comparable	Median Δ	Median token Δ	Success C $\rightarrow$ K	Quality Δ (pp)
DeepSeek V4 Flash	121/136	-32.7%	-188	129 $\rightarrow$ 131	+1.4
Laguna S 2.1	119/136	-29.2%	-156	129 $\rightarrow$ 131	+1.4
MiniMax M3	126/142	-33.7%	-207	135 $\rightarrow$ 136	+0.7
GLM 4.7 Flash	138/159	-29.3%	-160	154 $\rightarrow$ 152	-1.3
GLM 5.1	123/142	-36.7%	-202	140 $\rightarrow$ 141	+0.7
DeepSeek V4 Pro	128/145	-31.8%	-180	138 $\rightarrow$ 139	+0.7

Table 4: Retry-augmented leave-one-route-out sensitivity. Token summaries use equal-horizon pairs; quality uses all assigned pairs on the remaining routes (138, 138, 144, 159, 144, and 147, respectively). C/K denote clean/compact.

Across comparable pairs, 29/33 shorten, with median -57.1%. Both arms solve 33/36 continuations under the official suite. Among successful DeepSeek branches, extended submit-turn reverification, defined after acquisition as at least 350 reasoning tokens, occurs in 11/18 clean branches (61.1%) and none of the compact branches. Median final-turn reasoning falls from 489 to 12 tokens in the selected screen.

Across successful branches, 32 clean and 33 compact submissions preserve the shared candidate unchanged. Three unsuccessful DeepSeek pairs repeatedly call the earlier candidate-recording action after deriving corrected code. These observations concern verification and termination after shared acquisition; they cannot establish general repair ability or distinguish harmless checking removal from lost uncertainty. Passing public tests alone is not evidence that all further verification is redundant.

C Retrospective Style Sweep and Frozen Prompts

C.1 Design and interpretation

To explore whether terser state records are associated with shorter future reasoning, ten outcome-known DeepSeek source histories each receive three fresh randomized continuation blocks un-

Route	Shorter	Median Δ	Success C $\rightarrow$ K
DeepSeek	15/18	-93.9%	18/21 $\rightarrow$ 18/21
Laguna	14/15	-38.5%	15/15 $\rightarrow$ 15/15
Combined	29/33	-57.1%	33/36 $\rightarrow$ 33/36

Table 5: Acquisition-selected coding screen. Source-clustered 95% intervals for the medians are [-97.4, -2.4]% (DeepSeek), [-58.3, -29.2]% (Laguna), and [-95.9, -19.0]% (combined). C/K denote clean/compact.

der five history styles. Relative to clean, median future reasoning changes are -24.5% for relaxed prose, -27.0% for compact full sentences, -30.1% for telegraphic states, and -37.4% for ultra-telegraphic states. Source-clustered 95% intervals are [-37.7, -12.5]%, [-31.6, -17.6]%, [-41.0, -23.8]%, and [-41.9, -23.0]%. Every arm has 100% exact success.

Only 6/30 continuation blocks are monotonic across all tiers. Adjacent median contrasts are -4.9%, -6.8%, and -9.6%, and two of their three intervals include zero. Across 30 continuation blocks from ten source histories, the median Spearman correlation between history word count and future reasoning tokens is 0.70. Ordered arm medians therefore support an exploratory association at the group level. Length, syntax, wording, and compressor provenance vary jointly.

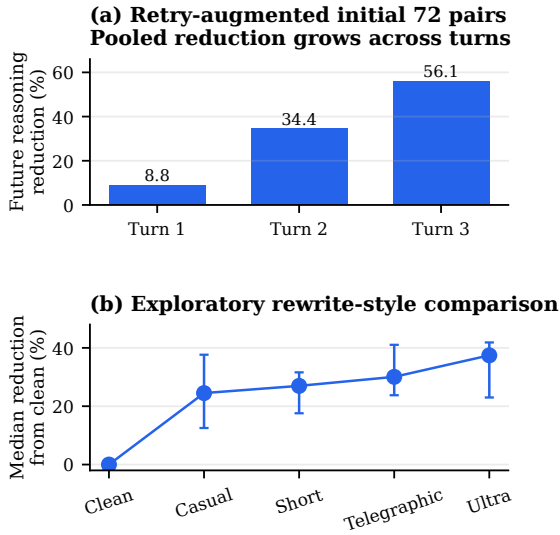


Figure 4: Distinct analyses of later reasoning. (a) Pooled reductions on the retry-augmented initial 72 pairs; later turns can include mediated effects. (b) Exploratory rewrite styles on outcome-known sources, with 30 continuations per arm. Ordered medians do not imply monotonic individual responses.

Figure 4 contains two distinct analyses. Panel (a) uses the retry-augmented initial 72 pairs and reports pooled token reductions of 8.8%, 34.4%, and 56.1% across turns. Panel (b) is the outcome-known style sweep. The two panels use distinct cohorts and estimands.

C.2 Frozen rewrite prompts

Several simple prompt formulations were tried during development. A model generated the frozen wording before evaluation; variants were not selected using measured compression, later reasoning, or task outcomes. The relaxed-prose, compact-full-sentence, and ultra-telegraphic treatments used the following common system prompt, paired with the exact style directive and turn-specific source fields.

Rewrite one historical assistant reasoning block into the requested style tier. Output only the rewritten state through the required JSON schema. Preserve every decision-relevant fact, number, unit, comparator, uncertainty, correction, rejected branch, evidence-status distinction, conclusion, dependency, and intended next action supported by the raw scratchpad and accepted compact anchor. The raw scratchpad is authoritative if the anchor adds process-control metadata; do not restore irrelevant repetition. Include the requested action's exact function name, argument keys, and argument values in the state. The action has not executed: do not add process metadata

saying its result is pending, withheld, absent, or forthcoming. Preserve an unknown/pending measurement only when the reasoning texts themselves state it. State every inequality direction and tie-break direction explicitly; 'limit X to N' or a bare metric name is not a substitute for $\leq/\geq$, lower/higher, at-most/at-least, minimum/maximum, or the source's exact direction. Never import a fact absent from both reasoning texts, improve the solution, use a later turn, observation, or final answer. Preserve opaque identifiers exactly; do not invent shorthand such as P2 for Phase 2 or DAY-21 for day 21. Stay inside the exact inclusive word interval. Do not emit template-control tags, code fences, labels, commentary, or an unexplained token list.

Relaxed or casual prose.

Express the state as relaxed, natural, complete prose with modest connective context. Remain materially shorter than the raw scratchpad. Never repeat facts merely to reach the interval.

Compact full sentences.

Express the compact state in ordinary complete grammatical sentences. Make dependencies explicit without adding explanation, repetition, or facts. Avoid telegraphic fragments and note headings.

Ultra-telegraphic states.

Write the shortest unambiguous state record possible. Use fragments, symbols, semicolons, and shared modifiers aggressively. Remove rhetoric, repeated rules, repeated arithmetic, headings, and prose that does not change the next decision. Omit no decision-relevant state.

The telegraphic treatment reused accepted parent rewrites. Nine source histories used this frozen system prompt:

Rewrite an assistant reasoning scratchpad into a shorter state record. Output only the rewritten scratchpad, with no preamble or code fence. Preserve every decision-relevant fact, number, uncertainty, conclusion, and requested next action. Do not add knowledge, improve the solution, or predict the requested tool's result: that tool has not run yet. Remove restatement, headings, rhetorical transitions, and repeated arithmetic. Fragments, equations, and compact clauses are allowed.

One reacquired source history used this variant:

Rewrite an assistant reasoning scratchpad into a shorter state record. Output only the rewritten scratchpad, with no preamble or code fence. Preserve every decision-relevant fact, number, uncertainty, conclusion, and requested next action. Preserve evidence-status distinctions exactly, including estimate versus binding quote and observed versus pending. Do not add knowledge, improve the solution, or predict the requested tool's result: that tool has not run yet. Remove restatement, headings, rhetorical transitions, and repeated arithmetic. Fragments, equations, and compact clauses are allowed.

D Continuous-Compaction Deployment Details

D.1 Assigned cohort and history policy

The fixed Verified Mini sample contains 25 Django and 25 Sphinx tasks, selected by an upstream size-optimized LP/clustering procedure. The dataset revision is b316c349947c29963fce3f4a65967c9807a4b673. We use upstream mini-SWE-agent 2.4.5 with a bash tool and 250-step contract, and the official SWE-bench 4.1.0 evaluator.

Before each target request, every unique prior nonempty reasoning block is compacted once and cached by its raw text and requested action name. Tool arguments and results remain outside the compressor payload. GPT 5.6 Luna/OpenAI at high effort is the external compressor; self-compaction uses the target model and route. Blocks of at most 12 words stay identical. A returned draft that is not shorter reverts to the raw block. Identifier and numeric omissions are recorded as lexical diagnostics. All tasks remain eligible; semantic failure requires separate assessment.

DeepSeek and MiniMax replay native reasoning history. Kimi and Nemotron use symmetric tagged-visible history in both arms because their tested transport lacks the same native replay contract. Those comparisons concern the symmetric visible representation.

The first model-terminal prediction is retained per task. Terminal statuses include submission, limit exit, repeated format error, and timeout. Infrastructure/provider failures may be retried, with completed calls in abandoned attempts still charged. Provider transitions retain already selected predictions. Nemotron acquisition moves from NVIDIA’s free endpoint to DeepInfra and BaseTen; Kimi uses Baidu, CoreWeave, and Chutes. These transitions make the deployment an operational policy comparison with serving variation.

D.2 Compression and selected tool calls

Main Table 2 reports deduplicated history words and selected-trajectory tool calls. Across eight treatment arms, compaction removes 1,651,881 of 2,606,959 unique raw words (63.4%). These counts omit repeated replay and non-reasoning context, so they cannot substitute for total billed input tokens. Tool calls count assistant-issued

calls in the terminal trajectory selected for evaluation, excluding compressor calls and abandoned attempts; their scope differs from acquisition-wide spend.

Tool-call analysis was defined retrospectively; neither target nor compressor was instructed to reduce tool use. In the four DeepSeek and MiniMax treatment arms, selected response counts fall 22.6–30.9%, mean reasoning per response falls 17.0–30.9%, and total reasoning falls 36.6–49.2%. All 50 selected DeepSeek trajectories submit. MiniMax submission counts are 38 under normal history, 42 under external compaction, and 44 under self-compaction, with correspondingly fewer limit or repeated-format exits. These describe changed iteration and termination. Their causal mechanisms remain unresolved.

Kimi’s increased tool use. To check the positive tool-call exception, we inspect all selected Kimi external-compaction trajectories. Calls rise from 1,990 to 2,384, a net increase of 394 (19.8%). They rise on 29 tasks, tie on three, and fall on 18; the paired median change is +9.3%. The three largest increases contribute 208 calls. Selected reasoning falls from 895,162 to 771,407 tokens (−13.8%), while assistant responses rise from 1,842 to 2,304 (+25.1%) and mean reasoning per response falls 31.1%. More responses with shorter reasoning account arithmetically for the aggregate pattern. No internal analysis or style-matching experiment identifies its cause.

D.3 Model-family compatibility hypothesis

One possible explanation for target-specific compaction effects is familiarity with the reasoning styles encountered during training. If a model’s reasoning training relies mainly on self-generated traces or distillation from teachers in the same family, it may encounter limited diversity in how reasoning states are expressed. Under this hypothesis, self-compaction could preserve familiar conventions, while a different family’s rewrites could change the cues used to resume reasoning and decide whether further checking is needed.

The Kimi deployment motivates this question: self-compaction incurs less total spending and fewer selected tool calls than GPT 5.6 Luna compaction (Table 2). The external arm resolves slightly more tasks in this sample, so this pattern concerns operational efficiency. We do not measure training-data diversity or teacher provenance.

Rewrite length, information retention, compressor capability and reasoning effort, caching, and provider variation also remain possible explanations.

Testing style compatibility requires crossing target models and compressors from the same and different families, with matched rewrite lengths, independently assessed state retention, and controlled inference settings. Content-matched stylistic variants could help separate familiarity from information loss. Testing the training-data explanation itself would require known teacher provenance or models trained on controlled mixtures of single-family and diverse-family reasoning traces. The present results do not establish this mechanism or any target’s training composition.

D.4 Paired task outcomes and uncertainty

To quantify uncertainty without treating repeated configurations as independent tasks, an offline audit resamples the 50 task IDs as pairs within each model–compressor comparison. Table 6 gives 10,000-draw paired-bootstrap intervals and discordant resolved sets. A gain is resolved only under compaction; a loss is resolved only under normal history. Intervals are conditional on the fixed selected sample and one captured run per arm, omitting unobserved seed and provider variation. These few discordant tasks do not establish preservation or improvement. Percentile intervals for arms with only gains or only losses have one-sided support and cannot sample an absent direction of discordance. Repeated appearances of tasks across configurations are not pooled as independent observations.

D.5 Request timing and spend provenance

To measure acquisition work independently of trajectory persistence, we build a response-level ledger from all captured acquisition journals and reconcile it with saved trajectories in both directions. Completed JSON responses with a nonempty choices array count as model responses for both target and compressor. Responses with identifiers are deduplicated by identifier; missing identifiers use the captured response hash. Completed JSON error bodies without model choices are excluded from primary duration sums and their additional captured time is reported separately. Unreturned requests and HTTP failures outside these completed records are not included.

Client-observed model-response duration includes queueing, prompt prefill, reasoning and answer generation, and network transfer. Inclusive time adds target and compressor durations across all acquisition attempts, including completed responses absent from saved trajectories. It excludes local tools, orchestration, and official evaluation; concurrency also separates summed request duration from elapsed wall time. Main Table 2 reports target-only time; Table 7 adds rewriting and gives absolute totals and inclusive changes relative to Normal. No complete rewrite-inclusive end-to-end wall-time comparison exists for this matrix. Different targets and acquisition histories account for the wide baseline spread; it is not a controlled comparison of serving speed.

Journal reconciliation adds 122, 420, and 254 Kimi target responses and 98, 205, and 72 Nemotron target responses absent from any saved trajectory, in normal/external/self order. Their recorded work remains in the totals even when task linkage is unavailable. No selected terminal trajectory or official task outcome is changed by this accounting procedure.

Common-tariff Nemotron estimates. To avoid comparing one arm’s free quota with another’s paid acquisition, all completed Nemotron target calls are repriced at common DeepInfra tariffs: \$0.50 per million uncached input tokens, \$0.10 per million cache-read tokens, and \$2.20 per million output tokens. Recorded cached tokens receive the cache-read rate; all recorded cache-write counts are zero. Actual compressor charges remain included. Normal, external, and self totals are \$23.82, \$38.33, and \$49.95, respectively. Thus the estimated increases are 60.9% and 109.7%.

The price normalization does not remove serving or retry confounding. Our monetary interpretation uses recorded input, cache, and output usage.

E Preprocessing and Marginal Latency

E.1 Hosted one-use and reuse accounting

To determine whether reduced continuation time recovers preprocessing overhead, a separate analysis uses the original initial-study captures: 69 comparable pairs from 24 sources, before the retrospective retries used in main Table 1. This equal-horizon two-route accounting example is separate from the full six-route workload comparison.

Continuation-only wall time falls 25.9% at the paired median (95% source-clustered CI

Target	Compressor	Gain/loss	Accuracy Δ (pp)	Paired-bootstrap 95% CI (pp)
DeepSeek V4 Flash	External	1/1	0	$[-6, 6]$
	Self	0/3	-6	$[-14, 0]$
MiniMax M3	External	3/2	+2	$[-6, 10]$
	Self	3/3	0	$[-10, 10]$
Nemotron 3 Ultra	External	3/0	+6	$[0, 14]$
	Self	2/2	0	$[-8, 8]$
Kimi K2.6	External	4/3	+2	$[-8, 12]$
	Self	3/3	0	$[-10, 10]$

Table 6: Paired resolution differences on 50 assigned tasks per configuration. The intervals describe task resampling with each captured run held fixed.

Target	Arm	Spend (\$)	Target (h)	Rewrite (h)	Inclusive (h)	Inclusive Δ (%)	Error (s)
DeepSeek V4 Flash	Normal	2.70	3.76	0.00	3.76	—	0.00
	External	1.60	2.24	1.25	3.49	-7.2	0.47
	Self	1.52	2.20	4.50	6.70	+78.3	0.00
MiniMax M3	Normal	23.68	42.18	0.00	42.18	—	0.00
	External	13.93	11.65	1.11	12.75	-69.8	1.36
	Self	11.58	9.91	1.57	11.48	-72.8	0.00
Nemotron 3 Ultra	Normal	23.82	7.97	0.00	7.97	—	95.24
	External	38.33	7.87	3.68	11.56	+45.0	20.91
	Self	49.95	10.22	7.30	17.52	+119.8	132.85
Kimi K2.6	Normal	48.84	31.07	0.00	31.07	—	41.74
	External	45.11	24.34	4.17	28.51	-8.2	253.15
	Self	33.26	13.10	10.33	23.43	-24.6	0.00

Table 7: All-acquisition spend and completed model-response duration sums, including abandoned attempts and responses absent from trajectories. Inclusive includes target and compressor time; its change is relative to Normal. Error gives additional captured JSON error-body time in seconds, excluded from the hour columns; those bodies have zero reported charges. Tools, orchestration, and evaluation are excluded. Nemotron spend uses common tariffs. Columns are rounded independently.

$[-38.4, -16.4]\%$), an absolute paired median difference of -2.825 seconds. Adding one source rewrite serially yields a 27.0% increase and a $+4.079$ -second paired median difference. Allocating its duration equally across three continuations gives a 5.6% decrease and a -0.657 -second paired median difference. These absolute and relative medians are computed separately and should not be combined to reconstruct a common baseline median.

Rewriting costs $\$0.339$, compared with $\$0.096$ for all initial controlled continuations. Every one-use case is more expensive; amortization over three replicates also fails to save money. The median added cost is $\$0.012906$ per one-use case and $\$0.004223$ under three-way allocation. These expensive controlled rewrites serve the intervention study; the deployment uses a different external compressor.

For a reusable history, a simple fixed-work accounting model would require $n(T_C - T_K) > T_R$ to recover rewrite time, and separately $n(C_C - C_K) > C_R$ to recover rewrite cost. Here T_R, C_R are one-time rewriting overheads and T_C, T_K, C_C, C_K are matched clean/compact continuation quantities. A finite break-even requires a positive per-use saving and assumes it persists under reuse. The reported medians and aggregate charges do not establish a universal break-even count, especially for money. The three-replicate allocation models experimental reuse.

A separate source-matched reanalysis uses all three continuations from each of 24 histories to estimate the mean marginal saving per source. Time savings are positive for 22/24 sources. Among those, the continuous time break-even threshold has median 1.76 continuations and range 0.43–8.33; 18/24 sources recover rewriting time by three uses. The other two have no finite time break-even under their observed means. Monetary marginal savings are positive for all 24, but the corresponding threshold has median 82.4 continuations and range 31.2–82,725; none recovers rewriting cost by three uses. These are source-conditional projections under fixed mean savings.

E.2 Single-task self-hosted marginal timing

To test whether reduced generation can translate into lower latency on a fixed serving stack, DeepSeek V4 Flash NVFP4 continues the SWE-bench task sphinx-doc__sphinx-10466 from already available clean, external, and self-compact

Regime (background)	Clean (s)	External Δ	Self Δ
Dedicated (0)	45.821	-26.8%	-25.4%
Shared (20)	73.464	-8.6%	-25.4%
Overloaded (61)	154.469	-26.1%	-23.4%

Table 8: Marginal postfork task time after compact history is available: five dedicated, five shared, and 23 complete original overloaded blocks. Clean is the regime median; treatment entries are paired median changes. Three experimental clients run in addition to the background clients.

histories on one B300 with TP1 and vLLM 0.25.0. Five dedicated and five shared three-arm blocks constitute the original test. A separate stress test attempts 25 three-arm blocks with 64 users and at most 32 active sequences. The primary timing comparison uses the 23 complete original blocks. Normal block two and self-compaction block 17 fail protocol checks; their original failures and durations remain in the record. A sensitivity analysis replaces those two individual arms with later recovery runs. Its 25 rows combine original arms with two noncontemporaneous replacements.

Across ten original blocks, self-compaction shortens every continuation’s time (paired median -25.4%), and external compaction shortens 90% (median -18.5%). Among the 23 complete original stress blocks, median changes are -23.4% for self and -26.1% for external compaction. The 25-row retry-augmented sensitivity gives -23.4% and -26.4% , respectively. Completed original branches have recorded resolved evaluations. The normal recovery lacks its evaluator-report file and enters only the timing sensitivity. Table 8 reports the original complete-block comparisons.

Postfork time covers model requests, orchestration, tools, patch capture, and local validation. It excludes rewriting, setup, replay, and hidden evaluation. The result estimates marginal continuation latency conditional on a prepared history. Completion here means protocol completion; the timing comparison does not establish quality preservation. Because continuations perform different work, the regimes do not identify a load-response curve. Resampling these blocks measures variation among repeated continuations of the same task.